المدرسة الوطنية للعلوم التطبيقية - بني ملال

Ecole Nationale des Sciences Appliquées - Béni Mellal

École Nationale des Sciences Appliquées de Béni Mellal

COMPTE RENDU DE TRAVAUX PRATIQUES PYTHON

1ère Année – Filière Intelligence Artificielle et Cybersécurité
Année Universitaire 2025–2026

Achraf HAMOUDA
7 NOVEMBRE 2025

Sous l'encadrement de : Mme. Ibtissame Aouraghe

INTRODUCTION

Ce TP s'inscrit dans le cadre du module de programmation Python, dont l'objectif est de nous initier à la logique algorithmique à travers un langage moderne, lisible et puissant, largement utilisé dans les domaines de l'intelligence artificielle, de la cybersécurité et de la science des données.

Dans ce TP 4, nous avons appris à utiliser les bases du langage Python.
L'objectif est de mieux comprendre comment fonctionnent les chaînes de caractères, les listes, les tuples et les ensembles (sets).

Pour la réalisation de ce TP, nous avons utilisé Visual Studio Code (VS Code), un environnement de développement moderne et polyvalent qui offre plusieurs avantages.

Manipulation de chaînes de caractères

Exercice 1 : Manipulation de chaînes de caractères

Cet exercice a pour but de manipuler une chaîne de caractères à l'aide de plusieurs méthodes intégrées de Python.

```
exo1.py > ...
message = "Apprendre Python est amusant et utile !"

#  Mettre la première lettre de chaque mot en majuscule
maj = message.title()
print("Première lettre de chaque mot en majuscule :",maj)

# Trouver combien de fois le mot 'et' apparaît
nbr_et = message.lower().count("et")
print("Nombre d'occurrences de 'et' est :", nbr_et)

# Vérifier si la chaîne commence par 'Apprendre'
debut = message.startswith("Apprendre")
print("La phrase commence par 'Apprendre' :", debut)

# Inverser la casse des lettres
inv = message.swapcase()
print("Casse inversée :",inv)
```

Figure 1: code source

- On crée une variable message contenant la phrase : "Apprendre Python est amusant et utile !".
- On utilise title() pour mettre la première lettre de chaque mot en majuscule.
- On compte combien de fois "et" apparaît grâce à count("et").
- La méthode startswith("Apprendre") permet de vérifier si la phrase commence par le mot Apprendre .
- Enfin, swapcase() inverse la casse des lettres, c'est-à-dire en majuscule ce qui était en minuscule et inversement.

Exécution1 :

```
/achra/Desktop/tp4_py/exo1.py
Première lettre de chaque mot en majuscule : Apprendre Python Est Amusant Et Utile !
Nombre d'occurrences de 'et' est : 1
La phrase commence par 'Apprendre' : True
Casse inversée : aPPRENDRE pYTHON EST AMUSANT ET UTILE !
```

Figure 2: run

Opérations sur les listes

Exercice 2 : Opérations sur les listes

Dans cet exercice, nous manipulons une liste de notes pour appliquer plusieurs opérations simples.

```python
exo2.py > ...
mylist = [14, 16, 12, 19, 18]

# L'ajout des nouveaux notes 17 et 13
mylist.append(17)
mylist.append(13)
print(f"La nouvelle liste est : {mylist}")

# Supression de la note la plus petite
a = min(mylist)
mylist.remove(a)
print(f"La nouvelle liste apres la suppression de la note minimal est : {mylist}")

# Calcul de moyenne
moy = sum(mylist) / len(mylist)
moyenne = round(moy,2)
print(f"La moyenne des notes est : {moyenne}")

# tri de la liste en ordre decroissant
mylist.sort(reverse=True)
print(mylist)
```

Figure 3: code source

- On crée la liste notes = [14, 16, 12, 19, 18].
- On ajoute deux nouvelles notes 17 et 13 avec la méthode append().
- On trouve la note la plus basse avec min(notes) et on la supprime à l'aide de remove().
- On calcule la moyenne en divisant la somme des éléments par leur nombre (sum(notes) / len(notes)).
- Enfin, on trie la liste en ordre décroissant avec sort(reverse=True).

Exécution2 :

```
/achra/Desktop/tp4_py/exo2.py
La nouvelle liste est : [14, 16, 12, 19, 18, 17, 13]
La nouvelle liste apres la suppression de la note minimal est : [14, 16, 19, 18, 17, 13]
La moyenne des notes est : 16.17
[19, 18, 17, 16, 14, 13]
PS C:\Users\achra\Desktop\tp4_py>
```

Manipulation de tuples et sets

Exercice 3 : Manipulation de tuples et sets

Cet exercice introduit deux structures de données : le tuple et le set .

```
exo3.py > ...
dimensions = (20,50,100)
print(dimensions[0])
# dimensions[0] = 1

set_a = {1,2,3,4,5}
set_b = {4,5,6,7,8}

# L'union des deux sets
a = set_a.union(set_b)
print(a)

# L'intersection des deux sets
b = set_a.intersection(set_b)
print(b)

# Les éléments de set_a qui ne sont pas dans set_b
c = set_a - b
print(c)
```

Figure 4: code source

- On crée un tuple dimensions = (20, 50, 100) et on essaie de modifier un élément. Python affiche une erreur, car un tuple est immuable.

- On crée deux sets :

set_a = {1, 2, 3, 4, 5} et `set_b` = {4, 5, 6, 7, 8}.

- On utilise les opérations d'ensemble :

union() pour réunit les deux ensembles ,

intersection() pour garde uniquement les éléments communs ,

difference() pour garde les éléments présents dans set_a mais pas dans set_b.

Exécution3 :

```
/achra/Desktop/tp4_py/exo3.py
20
{1, 2, 3, 4, 5, 6, 7, 8}
{4, 5}
{1, 2, 3}
PS C:\Users\achra\Desktop\tp4_py>
```

Méthodes des Listes

Exercice 4 : Méthodes des Listes

Cet exercice permet de pratiquer plusieurs méthodes courantes des listes en Python.

exo4.py > ...

```python
# Création de la liste
my_list = list(range(1,11,1))
print(my_list)

# Ajout de 11 et 12
my_list.append(11)
my_list.append(12)
print(my_list)

# Insertion de 0 au début
my_list.insert(0, 0)
print(my_list)

# Supprimer le dernier élément
my_list.pop()
print(my_list)

# Trouvants l'indice de 5
idx = my_list.index(5)
print(f"Indice de 5 : {idx}")

# Multiplier chaque élément par 2
for a in my_list :
    my_list[a] *= 2
print("Liste multipliée par 2 :", my_list)

##########################################

# Liste de fruits
fruits = ["pomme", "banane", "orange", "kiwi", "mangue"]

# Trier par ordre alphabétique
fruits.sort()
print("Fruits triés :", fruits)

# Ajouter un nouveau fruit
fruits.append("limouna")
print(fruits)
```

- On crée une liste de nombres de 1 à 10, puis on ajoute 11 et 12 avec append().
- On insère le nombre 0 au début de la liste grâce à insert(0, 0).
- On supprime le dernier élément avec pop().
- On cherche la position du nombre 5 avec index(5).
- On multiplie chaque élément par 2 à l'aide d'une boucle for.

- Dans la 2eme partie, on crée une liste de fruits, qu'on trie avec sort(), à laquelle on ajoute un fruit avec append(), puis on vérifie si "kiwi" est présent avec "kiwi" in fruits. Enfin, on supprime un fruit choisi par l'utilisateur avec remove().

Exécution4 :

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
Indice de 5 : 5
Liste multipliée par 2 : [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22]
Fruits triés : ['banane', 'kiwi', 'mangue', 'orange', 'pomme']
['banane', 'kiwi', 'mangue', 'orange', 'pomme', 'limouna']
'kiwi' est bien dans la liste.
Entrer une fruit a supprimer : limouna
Liste finale des fruits : ['banane', 'kiwi', 'mangue', 'orange', 'pomme']
PS C:\Users\achra\Desktop\tp4_py>
```

Slicing (Découpage)

Exercice 5 : Slicing (Découpage)

Cet exercice consiste à extraire certaines parties d'une liste à l'aide du slicing (opération de découpage par indices).

```python
exo5.py > ...
# Liste donnée
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# Afficher les éléments du 80 jusqu'au 10 en sens inverse
print(nums[7::-1])

# Afficher les éléments de 50 à 80
print(nums[4:8])

# Afficher un élément sur deux en partant de 90 vers la gauche
print(nums[::-2])

# Afficher les éléments à partir de 30, un sur deux
print(nums[2::2])

# Afficher la liste inversée complètement
print(nums[::-1])

# Afficher les trois derniers éléments 70, 80, 90
print(nums[6:9])

# En partant de 90, afficher un élément sur trois vers la gauche
print(nums[8::-3])
```

- Cet exercice consiste à faire des jeux de découpage sur nums.

Exécution5 :

```
[80, 70, 60, 50, 40, 30, 20, 10]
[50, 60, 70, 80]
[90, 70, 50, 30, 10]
[30, 50, 70, 90]
[90, 80, 70, 60, 50, 40, 30, 20, 10]
[70, 80, 90]
[90, 60, 30]
PS C:\Users\achra\Desktop\tp4_py>
```

Analyse d'une liste de phrases

Exercice 6 : Analyse d'une liste de phrases

Dans cet exercice, nous avons créé une fonction appelée analyser_phrases(phrases) qui permet d'analyser plusieurs phrases saisies par l'utilisateur.

exo6.py > analyser_phrases

```python
def analyser_phrases(phrases):

    #  Nettoyage + découpage
    liste_mots = []
    for p in phrases:
        p = p.strip()
        p = p.lower()
        mots = p.split()

        for m in mots:
            liste_mots.append(m)

    #  Ensemble des mots uniques
    mots_uniques = set(liste_mots)

    #  Total des mots
    total_mots = len(liste_mots)

    #  Total des mots uniques
    total_uniques = len(mots_uniques)

    #  Mot le plus long
    mot_plus_long = ""
    for m in mots_uniques:
        if len(m) > len(mot_plus_long):
            mot_plus_long = m

    #  Mot le plus fréquent
    mot_freq = ""
    max_count = 0
    for m in liste_mots:
        if liste_mots.count(m) > max_count:
            max_count = liste_mots.count(m)
            mot_freq = m

    #  Retourner les résultats
    return total_mots, total_uniques, mot_plus_long, len(mot_plus_long), mot_freq



phrases = []

n = int(input("Combien de phrases voulez-vous saisir ? "))

for i in range(n):
    phrase = input(f"Entrez la phrase {i+1} : ")
    phrases.append(phrase)

# afficher résultat
print(analyser_phrases(phrases))
```

- On définir une fonction appelée analyser_phrases(phrases).

Cette fonction prend une liste de phrases.

- Dans la première partie du code, chaque phrase est nettoyée :

on supprime les espaces inutiles au début et à la fin avec la méthode strip(),

puis on met toute la phrase en minuscules avec lower()

- Ensuite, chaque phrase est découpée en mots à l'aide de la méthode split().

Cette méthode transforme une phrase en une liste de mots séparés par des espaces.

- Tous les mots obtenus sont ensuite rassemblés dans une seule liste nommée liste_mots, grâce à une boucle for et la méthode append().

Cette liste contient donc tous les mots de toutes les phrases.

- On crée ensuite un ensemble (set) à partir de cette liste, appelé mots_uniques.

Un set permet de supprimer automatiquement les doublons, ie qu'un mot répété plusieurs fois n'apparaît qu'une seule fois dans cet ensemble.

- À partir de ces deux structures (liste_mots, mots_uniques), on calcule :

le nombre total de mots avec len(liste_mots),

le nombre de mots uniques avec len(mots_uniques),

le mot le plus long avec une boucle for qui compare les longueurs des mots,

et enfin, le mot le plus fréquent en comptant combien de fois chaque mot apparaît dans la liste avec count().

- Le programme retourne ensuite tous ces résultats sous forme de valeur, que l'on affiche ensuite dans le programme principal.

Exécution6 :

```
Combien de phrases voulez-vous saisir ? 2
Entrez la phrase 1 : HI my name Is ACHHHRAF
Entrez la phrase 2 : IACSSS sss
(7, 7, 'achhhraf', 8, 'hi')
PS C:\Users\achra\Desktop\tp4_py>
```

7 = nombre total de mots

7 = nombre de mots uniques

'achhraf' = mot le plus long

8 = longueur du mot le plus long

'hi' = mot le plus fréquent

FIN